**Sistema Preditivo Multiagente para Análise de Risco Hospitalar**

No contexto hospitalar surge a necessidade de criar um sistema inteligente, utilizando multiagentes para orquestrar e organizar uma fila de atendimento de pacientes.

Então, o CardioIA nessa fase implementa:

1. Um classificador supervisionado para a variável pico_risco;

2. Aplicação de 10 regras determinísticas;

3. Uso de quatro agentes orquestrados pelo OpenAI Agents SDK (handoffs reais) para conduzir a solução;

Foi usada a estrutura de dados propostos do enunciado. Então criada uma base de dados com 5000 registros sintéticos.

**2. Dados**

| Item | Descrição |
|---|---|
| Origem | `data/base_dados.py` (material oficial da aula) |
| Natureza | **100% sintética**, gerada com `np.random.seed(42)` |
| Registros | 5.000 |
| Variáveis de entrada | 10 (idade, carga do sistema, tempo de espera, FC, $SpO_2$, PA sistólica, temperatura, densidade de eventos, eventos em 24 h, disponibilidade de recursos) |
| Alvo | `pico_risco` ∈ {0, 1}, com **1.329 positivos (26,58%)** |
| Rotulagem | regra determinística: `risk_score ≥ 2`, onde `risk_score` soma 6 indicadores booleanos |
| Integridade | SHA-256 da base registrado no metadata do modelo |

Em seguida foram treinados dois modelos de classificação, um de Logistic Regression e outro de Random Forest Classifier (Árvore de decisão).

O critério de decisão foi o modelo que apresentasse maior recall.

**3. Metodologia de Machine Learning**

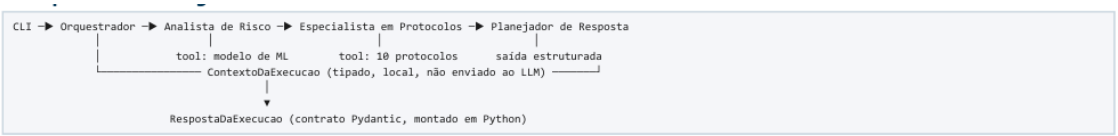
| Decisão | Adotada |
|---|---|
| Features | as 10 colunas originais, na ordem do arquivo oficial |
| Divisão | 80/20 **estratificada**, `random_state = 42` → 4.000 treino / 1.000 teste |
| Modelos comparados | `LogisticRegression` (pipeline com `StandardScaler`) e `RandomForestClassifier(n_estimators=300, random_state=42)` |
| Mesmos conjuntos | ambos treinados e avaliados sobre **o mesmo** treino e o mesmo teste |
| Critério de seleção | declarado **antes** de medir: maior *recall* da classe 1; empate por acurácia; depois interpretabilidade |
| Uso do teste | **uma única vez**, apenas para medir — sem ajuste de hiperparâmetros |
| Limiar de decisão | 0,5 — **operacional**, não calibrado clinicamente |
| Métricas | acurácia, *recall*, precisão, F1, matriz de confusão e relatório de classificação |

O melhor modelos, como esperado foi o Random Forest.

**4.1 Desempenho no conjunto de teste (mesmo para os dois modelos)**

| Modelo | Acurácia | Recall (1) | Precisão (1) | F1 (1) | Matriz de confusão `[[TN, FP], [FN, TP]]` |
|---|---|---|---|---|---|
| Regressão Logística | 0,8160 | 0,5526 | 0,6934 | 0,6151 | `[[669, 65], [119, 147]]` |
| **Random Forest** (escolhido) | **0,9990** | **0,9962** | 1,0000 | 0,9981 | `[[734, 0], [1, 265]]` |
| Regra determinística (teto) | 1,0000 | 1,0000 | 1,0000 | 1,0000 | `[[734, 0], [0, 266]]` |

Em seguida foi criada toda a estrutura, ambiente, agentes e tools da solução




| Agente | Responsabilidade | Não faz |
|---|---|---|
| **Orquestrador** | valida entradas, exige o artefato, inicia o run, monta/valida o contrato | não responde por conta própria |
| **Analista de Risco** | chama a tool que consulta o modelo salvo | não estima, não arredonda, não treina |
| **Especialista em Protocolos** | chama a tool que aplica as 10 regras oficiais | não cria regra nem altera limiar |
| **Planejador de Resposta** | consolida narrativa, justificativas e pontos de atenção | não recalcula nem introduz números |

**Resumindo**

o modelo de ML calcula a probabilidade de pico de risco, os protocolos em Python decidem as ações, e o LLM apenas conduz o fluxo.

**Os Agentes**

Orquestrador: valida o paciente, confere se o modelo existe e encaminha; monta a resposta final.

Analista de Risco: chama a tool que roda o modelo e traz a probabilidade real, sem arredondar.

Especialista em Protocolos: aplica as 10 regras do material e diz quais se acionam e por quê.

Planejador de Resposta: transforma o que já está no contexto em narrativa legível